Sentiment Classification of Restaurant Reviews

CS 273 Project 2.3

Yasaman Ebrahimi

***Abstract*—The goal of this project was to classify restaurant reviews as positive or negative using their text. I compared three machine-learning models with TF-IDF features: Logistic Regression, Multinomial Naive Bayes, and a linear Support Vector Machine (SVM). After cleaning the data, I used a shuffled and stratified 80/20 train-test split. Logistic Regression had the highest test F1-score of 0.868. The results also showed some overfitting because training performance was higher than test performance.**

I. Introduction

The main goal of this project was to see how well machine-learning models can predict the sentiment of a restaurant review. Positive sentiment was labeled 1, and negative sentiment was labeled 0. Although the same restaurant review dataset was used in Project 2.1, this project focused on sentiment rather than identifying fake and real reviews. I used only the review text to make predictions and only the sentiment column as the target.

II. Dataset Description

The original file, `restaurant_reviews-v2-1.xlsx`, contained 260 restaurant reviews. Each review had written text and labels for sentiment and whether it was fake or real. I selected the `Review` column as the input and `positive=1/negative=0` as the target. The fake/real label and the ten linguistic features from Project 2.1 were not used.

There were no missing review texts or sentiment labels. I removed four exact duplicate reviews before splitting the data to avoid having identical text in both sets. This left 256 reviews: 124 negative and 132 positive. The two sentiment classes were therefore close to evenly represented.

III. Data Preparation and Split

A. Train-Test Split

I used the `train_test_split` function from scikit-learn with `test_size=0.20`, `shuffle=True`, `stratify=y`, and `random_state=42`. Shuffling randomized the review order, and stratification kept the sentiment proportions similar in both sets. The final training set had 204 reviews, with 99 negative and 105 positive. The test set had 52 reviews, with 25 negative and 27 positive. The test reviews were not used when fitting the models.

B. Preprocessing

I kept the review wording mostly unchanged because the words and phrases themselves are important for sentiment. I used term frequency–inverse document frequency (TF-IDF) to convert the text into numerical features. The scikit-learn `TfidfVectorizer` handled lowercasing and tokenization and used both unigrams and bigrams with `ngram_range=(1,2)` and `min_df=2`. Bigrams can preserve short phrases such as "not good." I did not remove stopwords or apply stemming, since removing words such as "not" could change the meaning of a review.

Each classifier was placed in a scikit-learn pipeline with its own TF-IDF vectorizer. The vectorizer was fitted only on the training reviews, so vocabulary and feature weights were not learned from the test set. I used pandas to load and clean the dataset, scikit-learn to process text and train and evaluate models, and Matplotlib to create the comparison figures.

IV. Models and Training

I chose three traditional machine-learning classifiers because they work with the numerical features produced by TF-IDF and provide a simple comparison of different learning methods.

Logistic Regression is a linear classifier that estimates the probability of positive sentiment from the TF-IDF features. I used `LogisticRegression` with `max_iter=2000` and `random_state=42`.

Multinomial Naive Bayes is a probabilistic classifier commonly used with nonnegative text features. It estimates sentiment from nonnegative word features under a conditional independence assumption. I used scikit-learn's `MultinomialNB` with its default settings.

Linear SVM finds a separating boundary between the two sentiment classes by maximizing the margin. It is suitable for text represented by many sparse TF-IDF features. I used `LinearSVC` with `random_state=42` and its default settings.

All three pipelines were trained with `fit()` on the same 204 training reviews and their sentiment labels. I kept the model settings fixed in advance rather than tuning them on the test data. None of these models required a pre-trained transformer; they were trained directly for this binary task.

V. Evaluation and Results

I evaluated each trained model on both the training set and the untouched test set. Accuracy measures the proportion of correct predictions, while F1-score balances precision and

TABLE I
TRAINING AND TEST PERFORMANCE (THREE MODELS)

Model	Train Acc.	Train F1	Test Acc.	Test F1
Logistic Regression	1.000	1.000	0.865	**0.868**
Multinomial Naive Bayes	0.995	0.995	0.846	0.862
Linear SVM	1.000	1.000	0.846	0.840

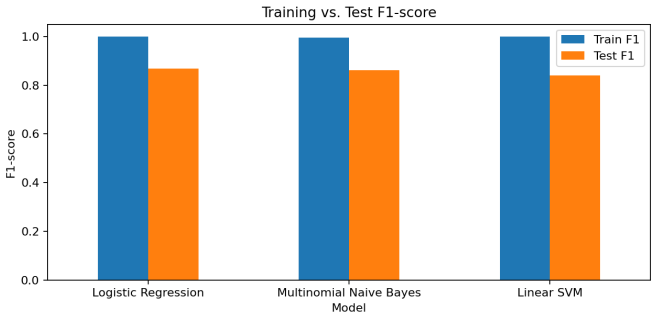


Fig. 1. Training and test F1-scores for the three models.

recall. For the equations below, TP and TN are true positives and true negatives, and FP and FN are false positives and false negatives. Positive sentiment (1) was treated as the positive class.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \tag{1}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}. \tag{2}$$

$$F_1 = \frac{2\,\text{Precision}\,\text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}. \tag{3}$$

Table I compares the accuracy and F1-score of all three models on both sets. Logistic Regression had the highest test F1-score (0.868) and test accuracy (0.865). Multinomial Naive Bayes was close behind, with a test F1-score of 0.862. Linear SVM had a test F1-score of 0.840. Naive Bayes and Linear SVM had the same test accuracy (0.846), but different F1-scores, which shows why it is useful to report both measures.

VI. Interpretation and Overfitting

As shown in Fig. 1, all three models performed better on the training reviews than on the test reviews. The train-minus-test F1 gaps were 0.132 for Logistic Regression, 0.133 for Naive Bayes, and 0.160 for Linear SVM. Logistic Regression and Linear SVM both scored 1.000 on training F1, while Naive Bayes scored 0.995. These gaps suggest that the models learned the training examples very well but did not perform as strongly on new reviews. The largest gap belonged to Linear SVM.

The confusion matrix in Fig. 2 shows that Logistic Regression correctly classified 22 negative and 23 positive reviews, for 45 correct predictions out of 52. It made three false positive and four false negative errors, which were nearly balanced.



Fig. 2. Test confusion matrix for Logistic Regression.

One limitation is the small dataset. With only 52 test reviews, a few changed predictions can noticeably affect accuracy and F1-score. The comparison also reflects one randomized split, so the results may change with different training and test examples. Using more labeled reviews and checking performance across several splits would give a better estimate of how the models perform on unseen reviews.

VII. Conclusion

This project compared three TF-IDF-based classifiers for predicting positive or negative restaurant review sentiment. The shuffled, stratified 80/20 split kept the classes close to balanced and allowed each model to be evaluated on separate test data. Logistic Regression performed best on this test set with an accuracy of 0.865 and an F1-score of 0.868. However, the difference between training and test results suggests some overfitting. The results show that these simple models can classify many reviews correctly, but more data would help evaluate how consistently they perform on new reviews.

VIII. Overleaf Project Link

https://www.overleaf.com/project/6ade71c302d657d9a9471ecd/share#91d5c902e3536a31eb891b94109a6fe45f742b0fb4e50e6d